



# **#42081 – Real-Time Operative Systems**

## ***Course Unit***

Basic concepts on real-time systems  
Requirements of Real-Time systems  
Real-Time Kernels/OSs

Paulo Pedreiras  
DETI/IT/UA  
SOTR, 2026/2027

# embedded and REAL-TIME Systems

## What are real-time systems?



- Computational systems
- Subject to the evolution of **REAL**-time:
  - **Do the right thing at the right time, otherwise things can go (very) wrong!**

# embedded and REAL-TIME Systems

## Is Real-Time that important?

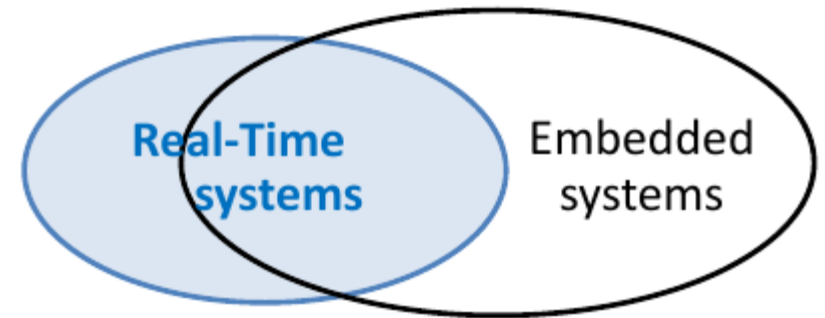
- It depends on the kind of system. If we're talking e.g. about control systems on a plane, a car's X-by-wire or a robot that moves around other people and equipment ... **Then YES!**



- On the other hand, if we are talking about multimedia systems or routers in home networks, delays in tasks shall not “cause death” (actual deaths or significant material losses, e.g. damages on expensive equipment) to anyone, but there will be a loss of Quality-of-Service.

# Embedded vs Real-Time systems

- **Real-time and embedded:**
  - Nuclear reactor control
  - Flight control
  - Mobile phone
  - Drone
- **Real-time, but not embedded:**
  - Stock trading system
  - Teams/Zoom
  - Video streaming/processing
- **Embedded, but not real-time:**
  - Digital camera
  - Printers, eReaders
  - Blood pressure meter

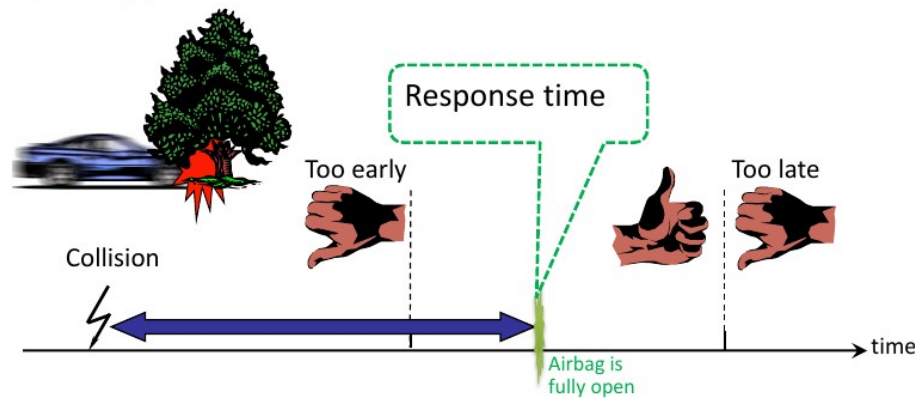


# A few definitions related with “Real-Time”

There is a wide variety of definitions related to **Real-Time**, **systems** dealing with Real-Time and the **services** that they provide.

All of these definitions have in common the fact that **express the dependency of a computer system on the time**, as it exists in a particular **physical process**.

Example: airbag system



[https://www.google.com/search?q=airbag+inflating+video&xsrf=APwXEdCHXFID-K58YQuY4y1475Suwng%3A1681136548196&ei=pB0ZMDIC4rasAe56aDYAQ&ved=0ahUKEwjAqjWwZ\\_-AhUKLewKHbk0CBsQ4dUDCA8&uact=5&oq=airbag+inflating+video&gs\\_lcp=Cgxnnd3Mtd2l6LXNlcnAQAzIGCAAQFhAeOgcIABAEELAD0gQIIxAnOggIABAWEB4QD0oEECEYAVDvDjJWH2CHJWgBcAB4A1ABlwG1AcEHkgEDMC44mAEoAEByAEBwAEB&scient=gws-wiz-serp#fstate=ive&vid=cid:05dc1b09,vid:90eA14Wzq7](https://www.google.com/search?q=airbag+inflating+video&xsrf=APwXEdCHXFID-K58YQuY4y1475Suwng%3A1681136548196&ei=pB0ZMDIC4rasAe56aDYAQ&ved=0ahUKEwjAqjWwZ_-AhUKLewKHbk0CBsQ4dUDCA8&uact=5&oq=airbag+inflating+video&gs_lcp=Cgxnnd3Mtd2l6LXNlcnAQAzIGCAAQFhAeOgcIABAEELAD0gQIIxAnOggIABAWEB4QD0oEECEYAVDvDjJWH2CHJWgBcAB4A1ABlwG1AcEHkgEDMC44mAEoAEByAEBwAEB&scient=gws-wiz-serp#fstate=ive&vid=cid:05dc1b09,vid:90eA14Wzq7)

# Real-Time Computation

The **computation results** must be

- Logically correct
- Delivered on time

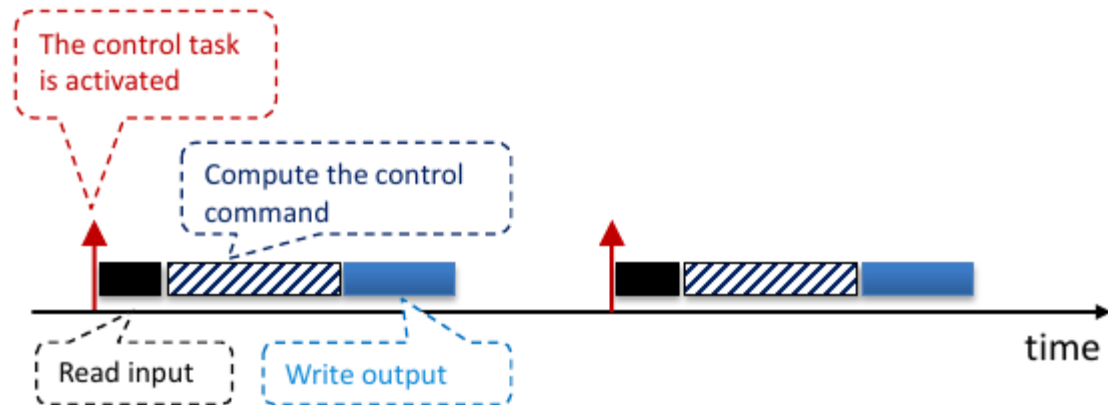
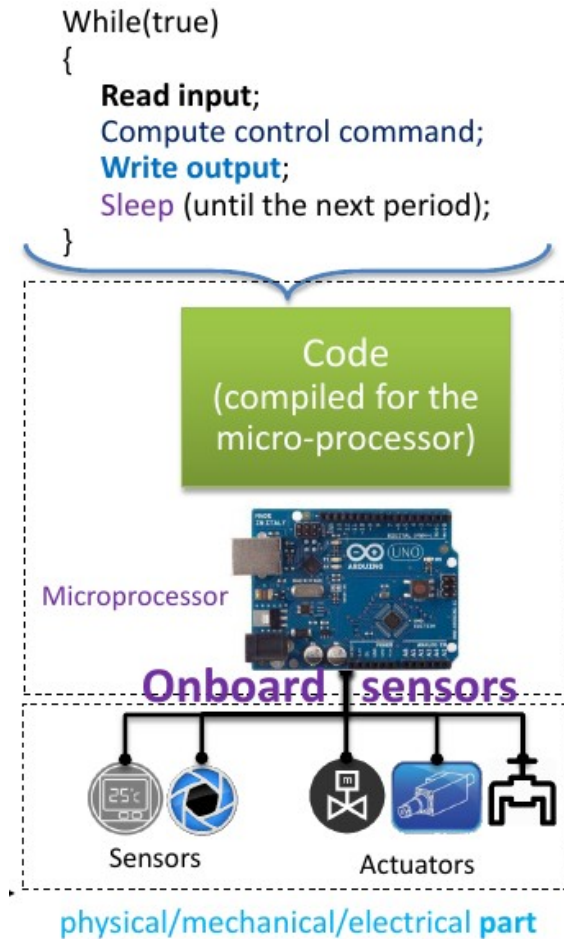
(Stankovic, 1988)

**Timeliness**



**Logic correction**

# Real-Time systems - implementation



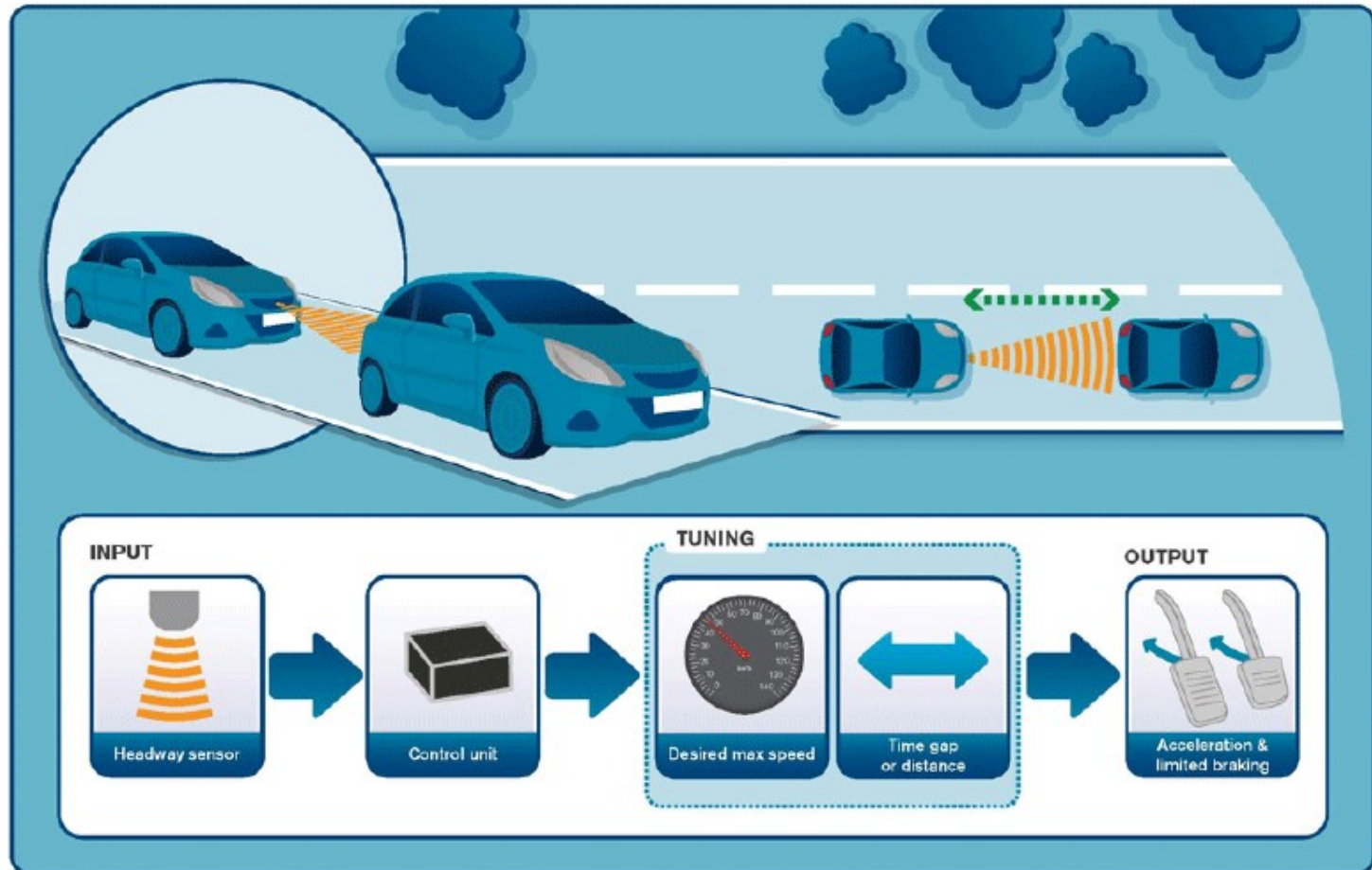
Simple case, with an infinite loop:

- Gather inputs, decision and action

**Looks simple and predictable, right?**

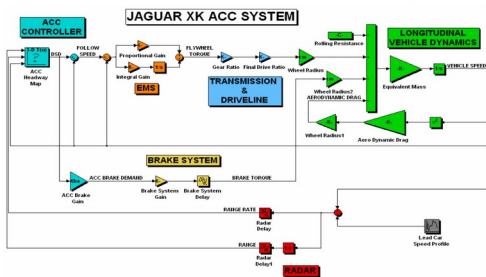
# Real-Time systems - implementation

## ACC Adaptive Cruise Control

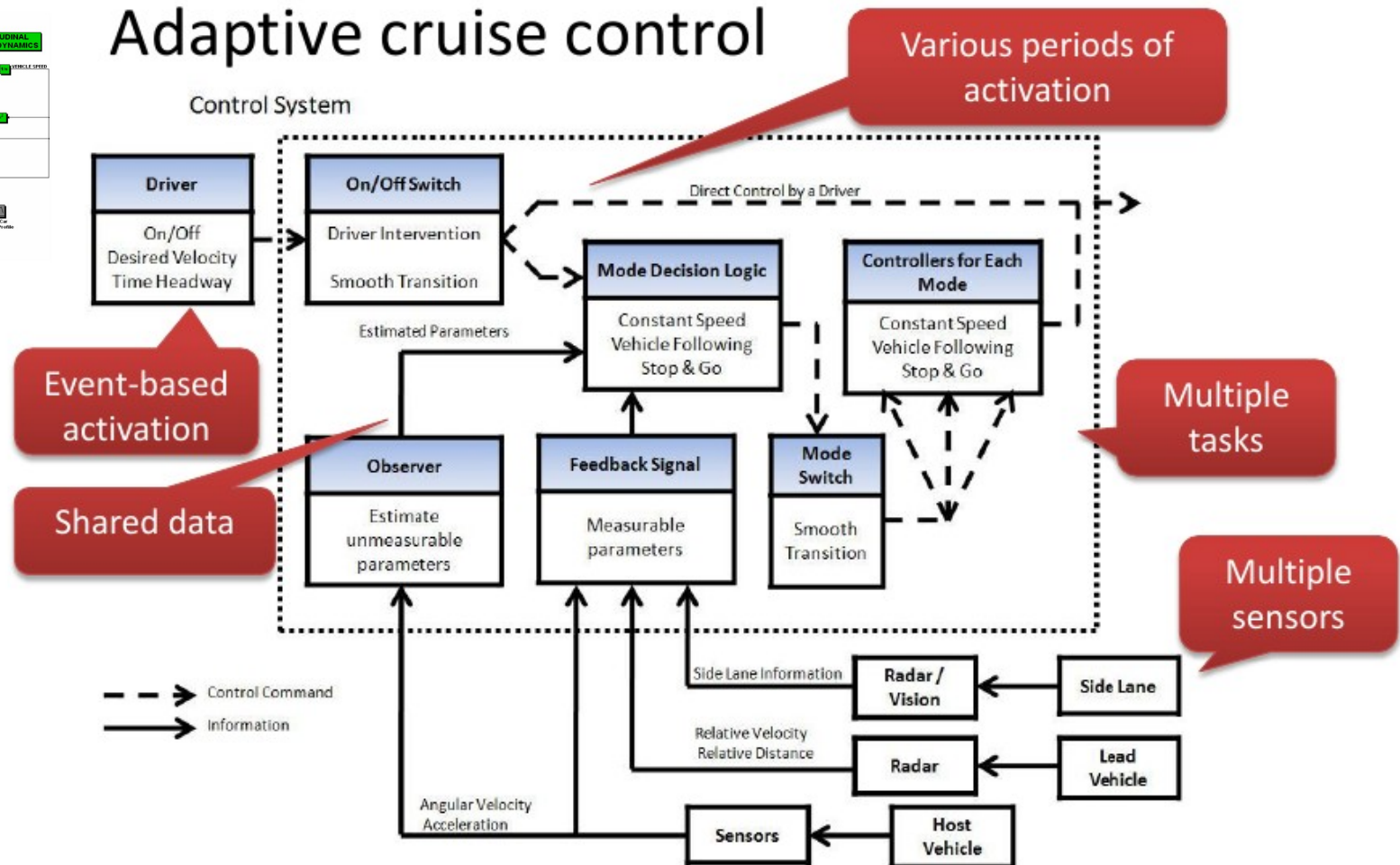


**But it can get much more complicated ...**

# Real-Time systems - implementation



## Adaptive cruise control



From: <https://codingcommunity.de/jaguar-cruise-control-diagram.html>

Does it still look predictable?

# Requirements of Real-Time Systems

The **requirements** commonly imposed to real-time systems are of three types:

- **Functional**: computations must be logically correct
- **Temporal**: meeting timing constrains
- **Dependability**: (in RT systems) the ability to provide services that can be trusted within a time-period

# Objective of the study of RTS

Regarding the computational activities of RTS, the main **Temporal** aspects to consider are:

- **Execution time**: how much time **an activity/task** takes to complete (in isolation)
- **Response time**: how much time each activity/task of a given **set of activities/tasks** takes to complete
- **Regularity of periodic events**: important e.g. for control applications

# Objective of the study of RTS

## Some aspects particularly important regarding

- Execution time
  - Code structure (language, conditional execution, cycles)
  - DMA, cache, pipeline
  - Operative System or kernel (system calls)
- Response time and regularity
  - Interrupts
  - Multi-tasking
  - Access to shared resources (buses, communication ports, ...)

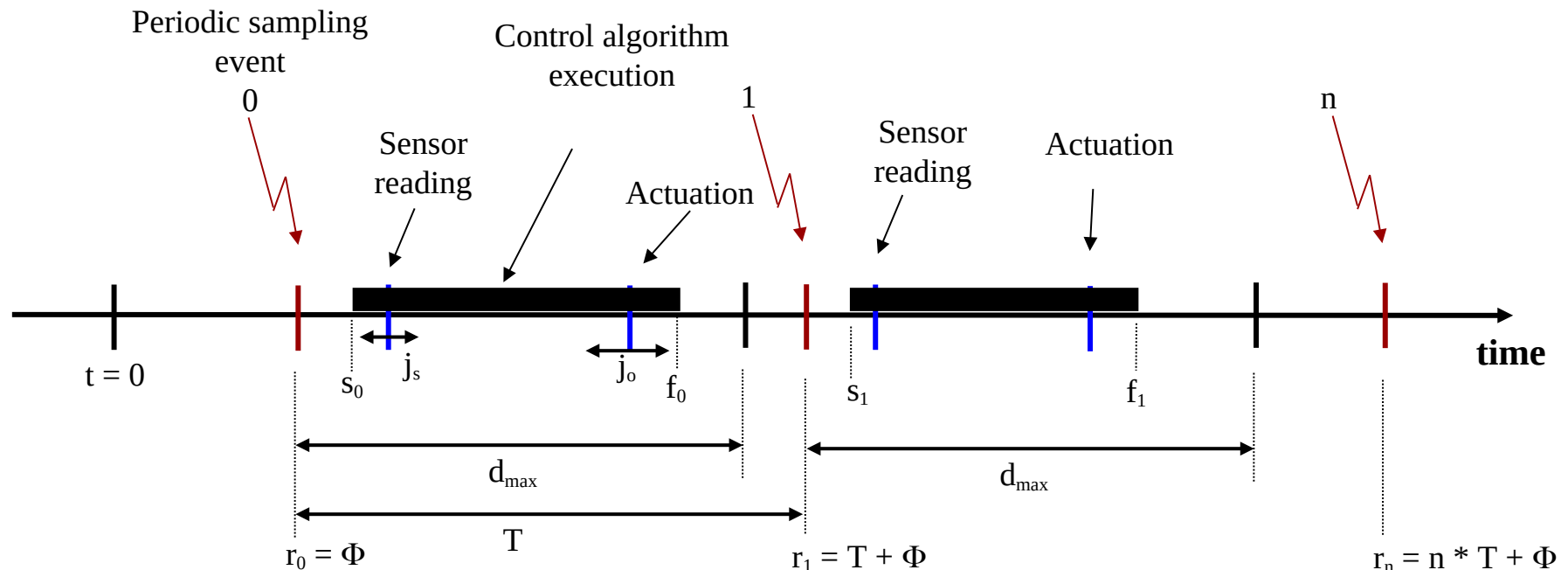
# Temporal requirements

- **Temporal requirements** usually arise from the **physical dynamics** of the process to be controlled .
- Impose restrictions:
  - **Delays on the observation** of the system state
  - **Delays on computing** the new control values (acting)
  - **Variations** of previous delays (jitter)
- These restrictions must be **respected all instances** (including the worst case) and not only on average

# Temporal requirements characterization

Most common temporal requirement is **deadline**:

- Maximum amount of time that a task can take to complete, with respect to its activation/release



$r_n$  – activation (release)

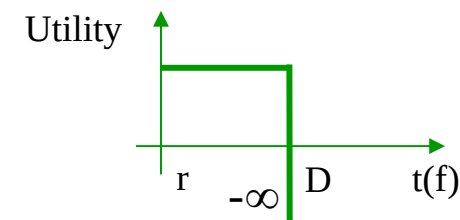
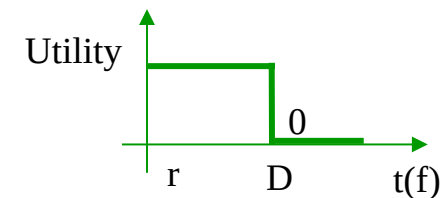
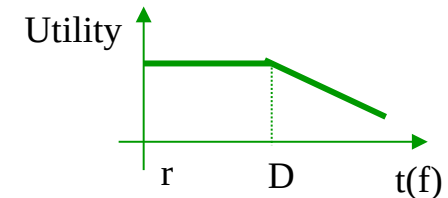
$s_n$  – start of execution

$f_n$  – end of execution

# Temporal requirements

Classification of the temporal constraints, according to the **usefulness of the result**:

- **Soft** – temporal constraint in which the result retains some utility to the application, even after a temporal limit  $D$ , although affected by a degradation of quality of service.
- **Firm** – temporal constraint in which the result loses any usefulness to the application after a temporal limit  $D$ .
- **Hard** – temporal restriction that, when not met, can lead to a catastrophic failure.



# Temporal requirements

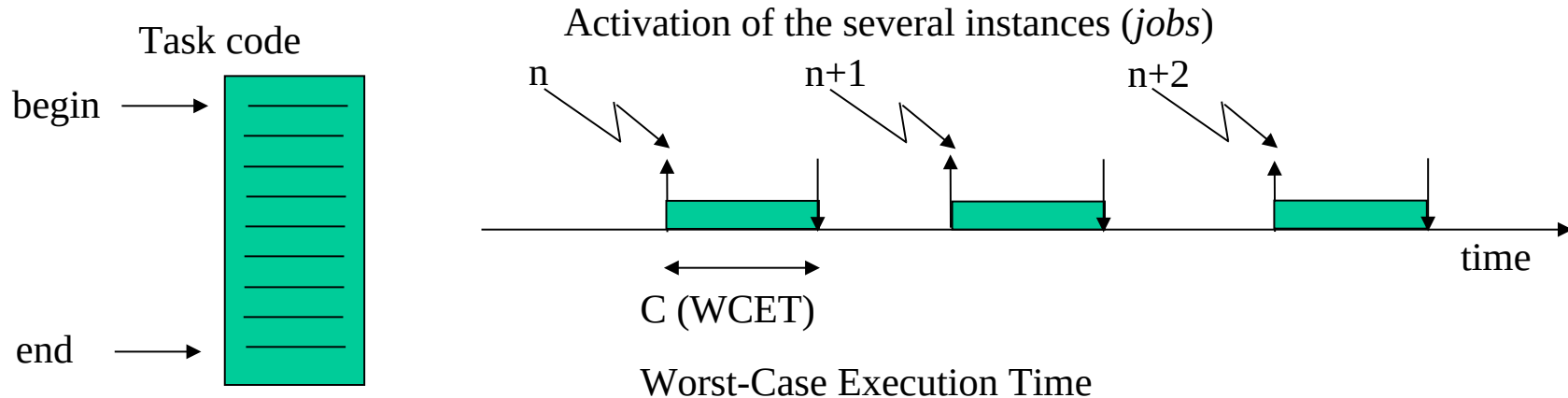
Classification of the Real-Time Systems, according to the **temporal constraints**

- **Soft Real-Time:** The system only has **firm or soft** real-time constraints (e.g., simulators, multimedia systems)
- **Hard Real-Time:** The system has **at least one hard** real-time constraint. These are the so-called safety-critical systems (e.g. airplane control, missile control, nuclear plants control, control of dangerous industrial processes)

# Real-time model

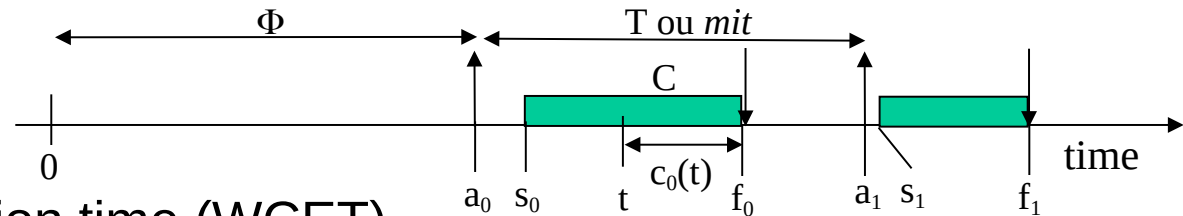
## Definition of task (aka process, thread, activity)

Sequence of activations (instances or jobs), each consisting of a set of instructions that, in the absence of other activities, is performed by the CPU without interruption.



# Real-time model

## Task characterization

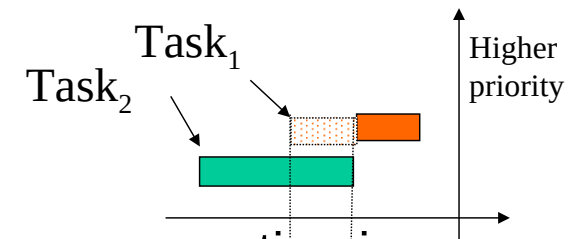
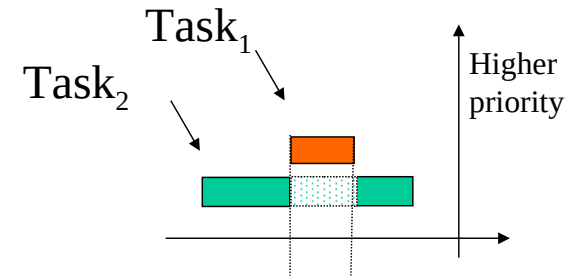


- **C** worst-case execution time (WCET)
- **T** period (if periodic)
- $\Phi$  relative phase = offset of 1st activation (if periodic)
- **mit** *minimum inter-arrival time* (if sporadic)
- $a_n$  activation instant of the  $n^{\text{th}}$  instance
- $s_n$  start time of the  $n^{\text{th}}$  instance
- $f_n$  finish time of the  $n^{\text{th}}$  instance
- $c_n(t)$  maximum residual execution time of the  $n^{\text{th}}$  instance at time  $t$

# Real-time model

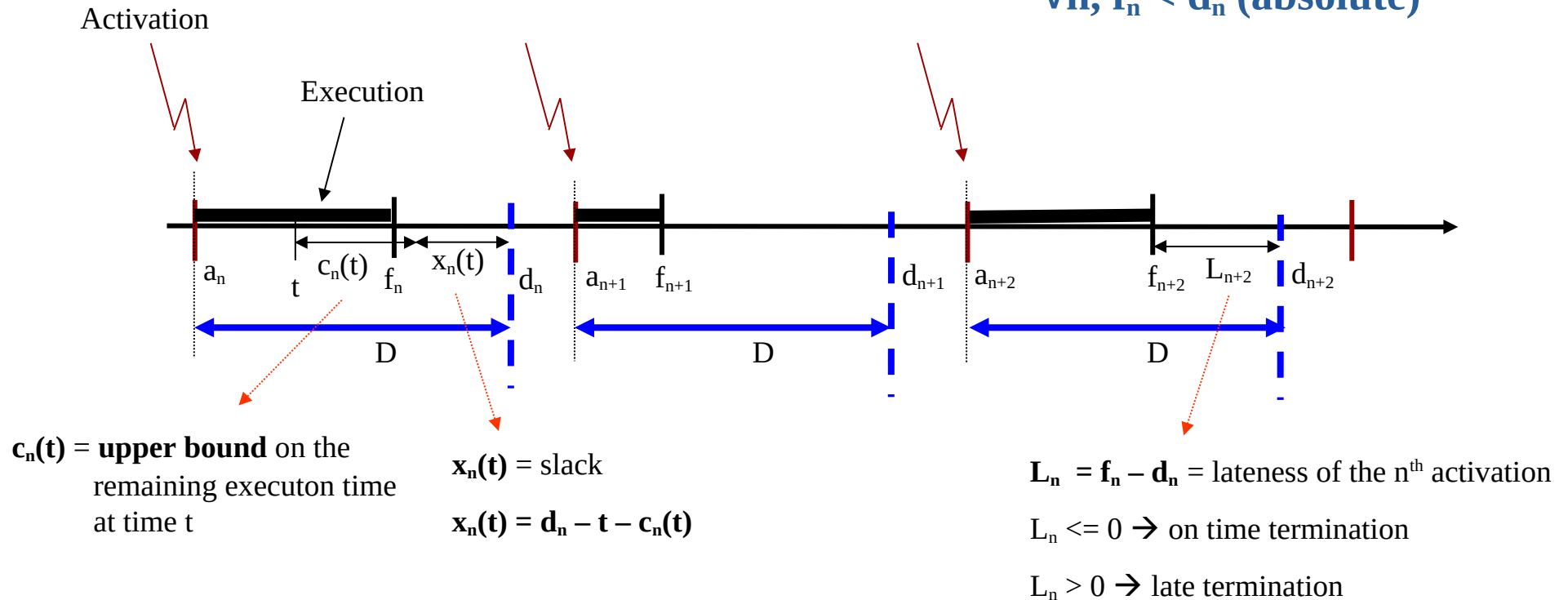
## Preemption:

- When a **task** can be interrupted temporarily for execution of other higher priority task, it is said that it **admits preemption**.
- When a **system** uses the preemption property of the tasks that it executes, it is designated **preemptive**.
- A set of tasks is said to admit **full preemption** when preemption is admitted by **all tasks at any point** in its execution (independent tasks)
- Note: access to **shared resources** (tasks with dependencies) may **restrict** the preemptiveness of tasks.



# Real-time model

**Deadline classification**  $\forall n, f_n - a_n < D$  (relative)  
 $\forall n, f_n < d_n$  (absolute)



- There are other types of task requirements besides deadlines: window, synchronization, distance, ...
- Deadlines are the most common type

# Real-time model

## Example of task characterization:

- **Periodic:**  $\tau_i = \tau_i (C_i, \Phi_i, T_i, D_i)$

$$\tau_1 = \tau_1 (2, 5, 10, 10), \tau_2 = \tau_2 (3, 10, 20, 20)$$

- **Sporadic:** Similar to periodic, but  $mit_i$  replaces  $T_i$  and  $\Phi_i$  usually is not used (though it could be used to specify a minimum delay until the first activation).

$$\tau_i = \tau_i (C_i, mit_i, D_i)$$

$$\tau_1 = \tau_1 (2, 5, 5), \tau_2 = \tau_2 (3, 10, 7)$$

# Temporal control

## Triggering of activities

- **By time (*time-triggered*)**
  - The execution of an activity (function) is triggered via a control signal based on the progression of time (e.g., through a periodic timer interrupt).
- **By events (*event-triggered*)**
  - The execution of activities (functions) is triggered through an asynchronous control signal based on a change of system state (e.g., through an external interrupt).

# Temporal control

Systems controlled by the occurrence of events on the environment

## *event-triggered (ET) systems*

- Typical of sporadic condition monitoring on the system state (e.g., alarm verification or asynchronous requests).
- **Utilization rate** of the the computing system resources (e.g. CPU) is **variable**, depending on the frequency of occurrence of events.
  - Ill-defined worst case situation. Implies either:
    - use of probability arguments, or
    - limitation on the maximum rate of events (sporadic systems)

# Temporal control

## Systems triggered by the progression of time

### *time-triggered (TT) systems*

- Typical e.g. in **control applications** of cyber-physical systems
- There is a **common time reference** (allows establishing phase relations)
- CPU **utilization is constant**, even when there are no changes in the system state.

**Worst case situation is well-defined**

# Temporal control

- Example:
  - For the following task sets compute the maximum delay that each task may suffer:
    - TT  $\{\tau_i = \tau_i (C_i=1, \Phi_i=i, T_i=5, D_i=T_i \ i=1..5)\}$
    - ET  $\{\tau_i = \tau_i (C_i=1, (\Phi_i=0), \text{mit}_i=5, D_i=\text{mit}_i \ i=1..5)\}$
  - Compute the average and maximum CPU utilization rate for both cases. Admit that, on average, ET tasks are activated every 100 time units.

Note: the CPU utilization is given by:

$$U = \sum_{i=1}^N \frac{C_i}{T_i}$$

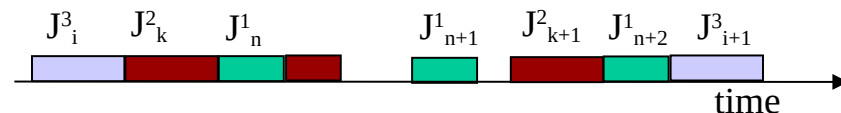
# Task states

- **Task creation**

- Association between executable code (e.g. a “C” language function) to a private variable space (private stack) and a management structure - task control block (TCB)

- **Task execution**

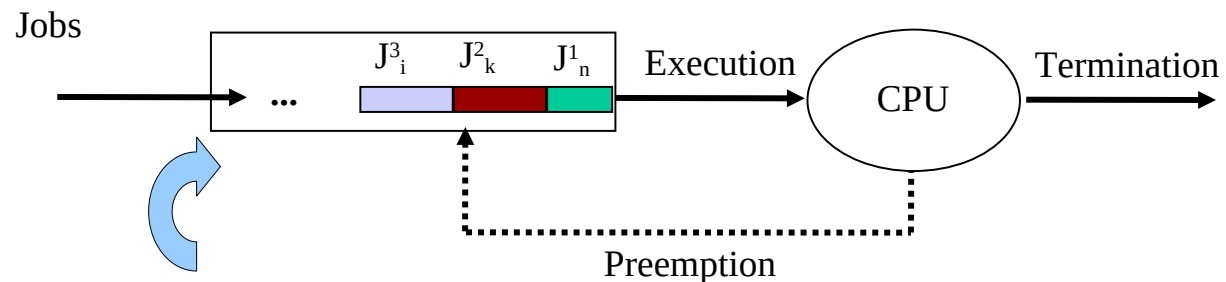
- Concurrent execution of the task's code, using the respective private variable space, under control of the kernel.
- The kernel is responsible for activating each one of the task's jobs, when:
  - A period has elapsed (periodic)
  - An associated external event has occurred (sporadic)



# Task states

## Execution of task instances (jobs)

- After being activated, task's jobs wait in a queue (the ready queue) for its time to execute (i.e., for the CPU)
- The ready queue is sorted by a given criterion (scheduling criterion). In real-time systems, most of the times this criterion is not the arrival order!

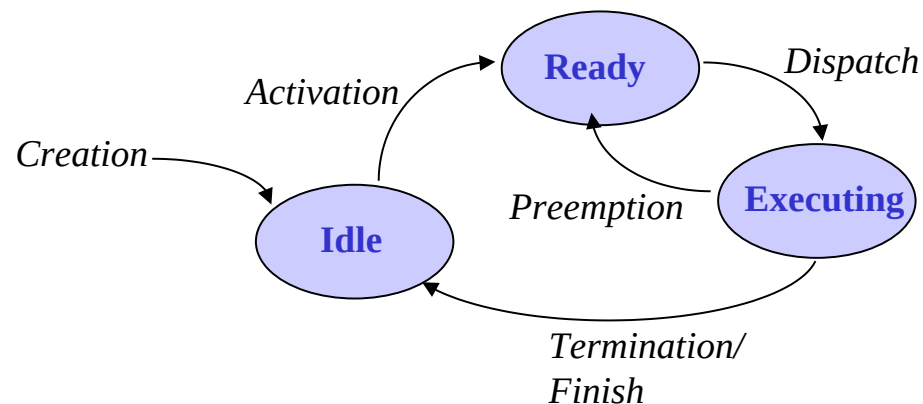


**Task sorting method defines the so called scheduling policy**

# Task states

## Task states

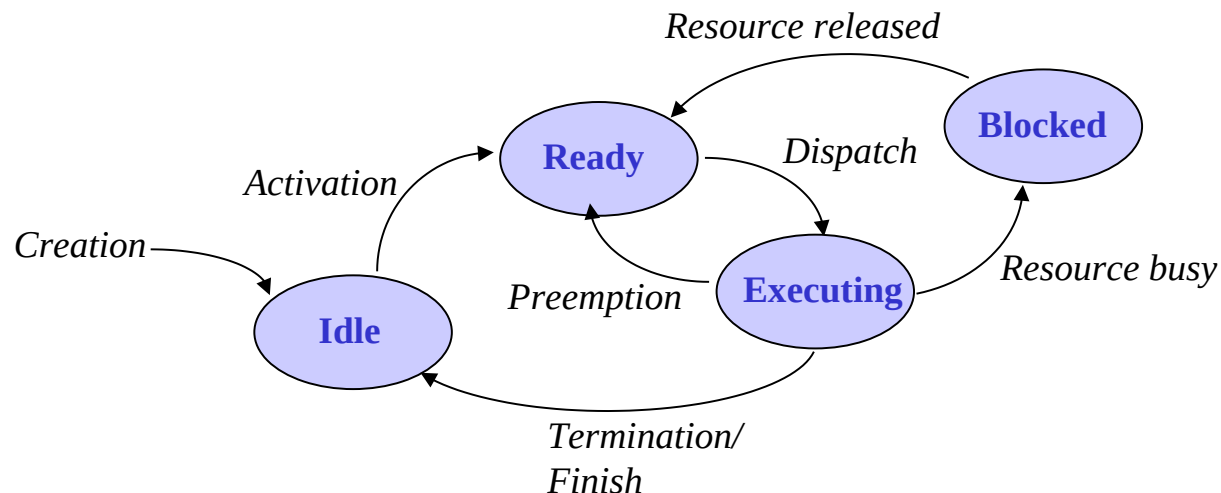
- Task instances may be waiting for execution (ready) or executing. After completion of each instance, tasks stay in the idle state, waiting for its next activation.
- Thus, the basic set of dynamic states is: **idle**, **ready** and **execution**.



# Task states

## Other states: blocked

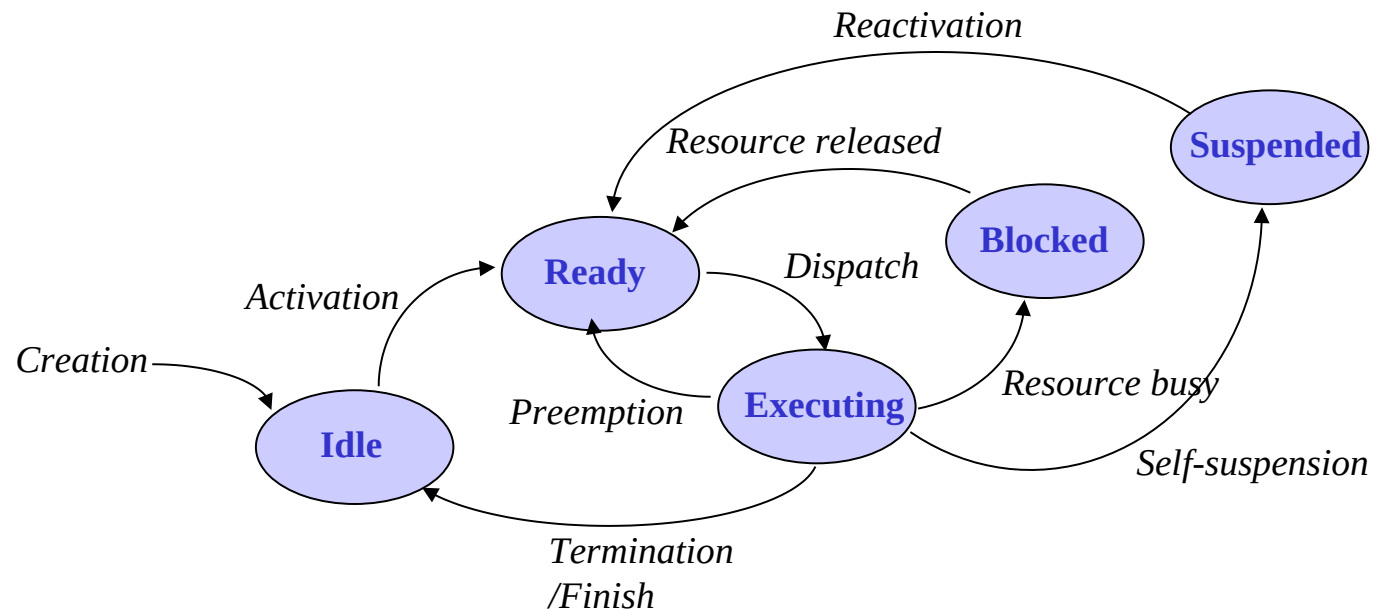
- Whenever an executing task tries to **use a shared resource** (e.g. a memory buffer) that is already being used in exclusive mode, the task cannot continue executing.
- In this case it is moved to the **blocked state**. It remains in this state until the moment in which the **resource is released**. When that happens the task goes to the ready state.



# Task states

## Other states: self suspension (sleep)

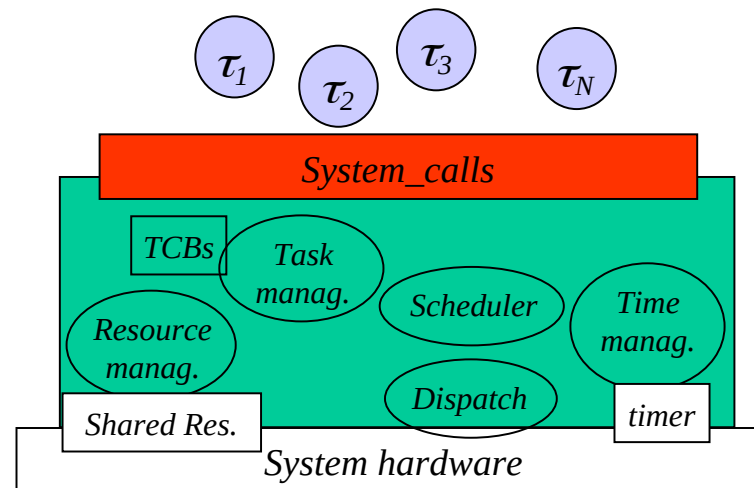
- In certain applications tasks need to **suspend** its execution for a given **amount of time** (e.g. waiting a certain amount of time after requesting an ADC conversion), before completing its execution. In that case tasks move to the suspended state.



# Internal Architecture of a Real-Time Kernel

## Basic services

- Task management: create, delete, initial activation, state
- Time management: activation, policing, measurement of time intervals
- Task scheduling: decide what jobs to execute in every instant
- Task dispatching: putting jobs in execution
- Resource management: mutexes, semaphores, etc.



# Management structures

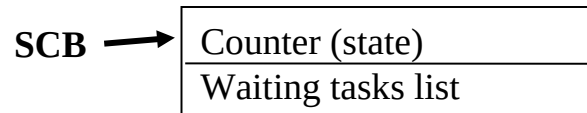
## The TCB (task control block)

- This is a fundamental structure of a kernel. It stores **all the relevant information about tasks**, which is then used by the kernel to manage their execution.
- Common data (not exhaustive)
  - Task identifier
  - Pointer to the code to be executed
  - Pointer to the private stack (for context saving, local variables, ...)
  - Periodic activation attributes (task type (periodic/sporadic), period, initial phase, etc)
  - Criticality (hard, soft, non real-time)
  - Other attributes (deadline, priority)
  - Dynamic execution state and other variables for activation control, e.g. SW timers, absolute deadline, ...

# Management structures

## Access to shared resources

- Exclusive access shared resources (critical sections) has to be managed in an appropriate way, to allow access by only one task at a time (as for the CPU).
  - A simple way to do it is using atomic flags/, monitors, mutexes or semaphores.
- For the case of semaphores it is needed a structure (**semaphore control block – SCB**) that holds its state as well as the list of tasks that are waiting for access.



# Management functions

## Time management

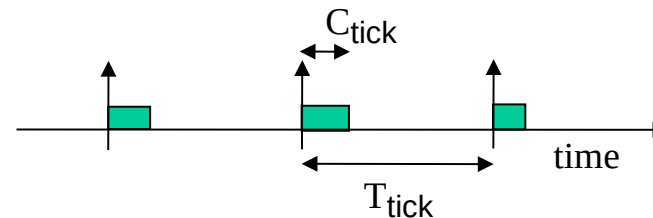
- Time management is another critical activity on kernels. It is required to:
  - Activate periodic tasks
  - Check if temporal constraints are met (e.g. deadline violations)
  - Measure time intervals (e.g. self-suspension)
- It is based on a system timer. This timer can be configured in two modes:
  - **Periodic tick:** generates periodic interrupts (system ticks). The respective ISR handles the time management. All temporal attributes (e.g. period, deadline, waiting times) must be integer multiples of the clock tick.
  - **Single-shot/One-shot/tickless:** the timer is configured for generating interrupts only when there are periodic task activations or other similar events (e.g. the termination of a task self-suspension interval).

# Management functions

## Tick-based systems

- The tick defines the system's **temporal resolution**.
  - **Smaller ticks corresponds to better resolutions.**
  - E.g. with a 10ms tick and task periods:  $T_1=20\text{ms}$ ,  $T_2=1290\text{ms}$ ,  $T_3=25\text{ms}$
- The tick handler is code that is executed periodically. Thus it consumes CPU time, representing overhead ( $C_{\text{tick}}/T_{\text{tick}}$ )
  - **The bigger the tick, the lower the overhead!!**
  - Compromise is needed:

$$\text{tick} = \text{GCD}(T_i, i=1..N)$$

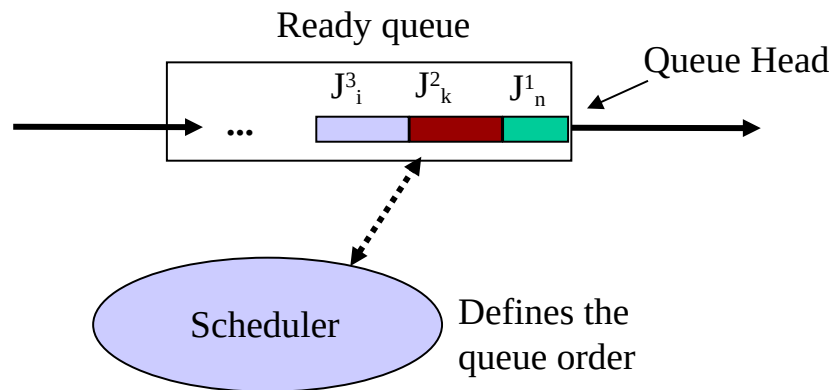


- Same e.g.  $T_1=20\text{ms}$ ,  $T_2=1290\text{ms}$ ,  $T_3=25\text{ms}$ , then  $\text{GCD}(20,1290,25)=5\text{ms}$
- It must be assured that  $\text{tick} > \text{min\_tick}$ , which is imposed by the CPU processing capacity!

# Management functions

## Scheduler

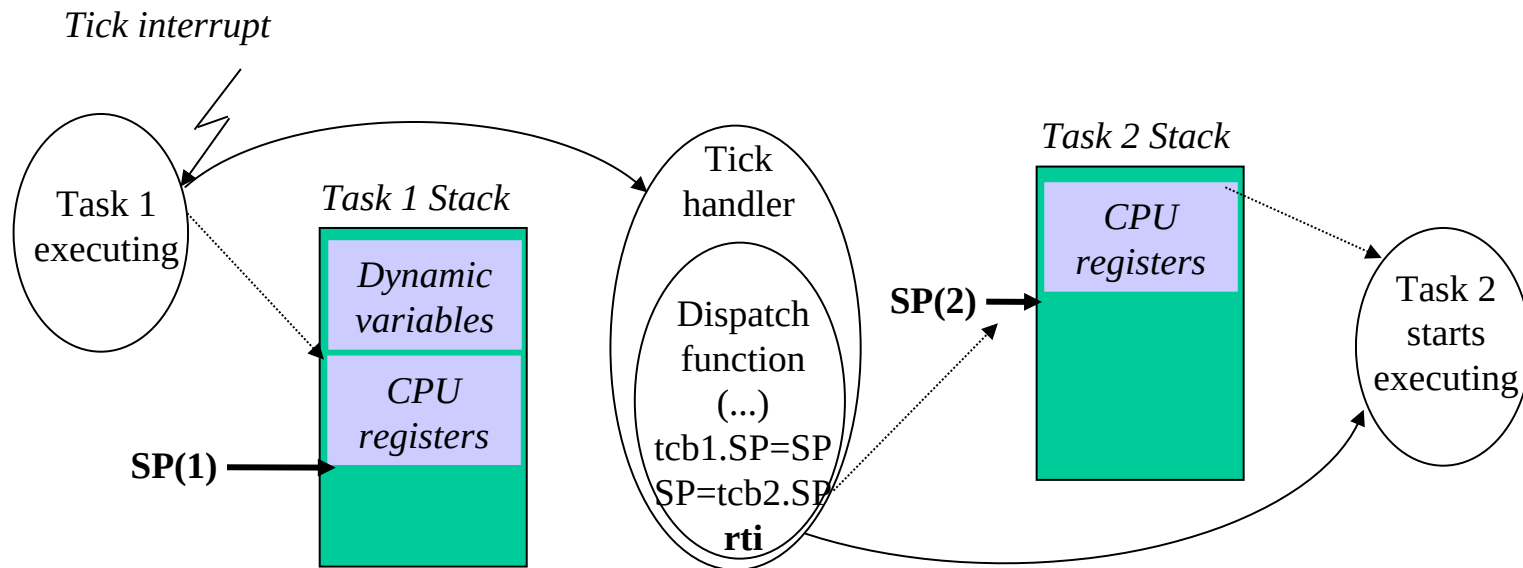
- The scheduler **selects which task to execute** among the (eventually) several ready tasks
- In real-time systems must be based on a **deterministic criteria**, which must allow computing an upper bound for the time that a given task may have to wait on the ready queue.



# Management functions

## Dispatcher

- Puts in execution the task selected by the scheduler
- For preemptive systems it may be needed to preempt (suspend the execution) of a running task. In these cases the dispatch mechanism must also manipulate the stack.



# Examples of RTOS

There are many RTOS in the market, some targeting small embedded microcontrollers while others aim at high-performance hardware.

Licensing conditions also variable (from FOSS to normal commercial licenses)

(Just) A few examples:

- **Zephyr:** addressed in the lab component
- **FreeRTOS:** owned by Amazon, aiming at IoT & embedded microcontrollers
- **Mbed OS:** also aims at IoT, owned by ARM and supports only ARM Cortex-M platforms
- **Azure RTOS ThreadX:** also aims at IoT and deeply embedded applications
- **WindRiver VxWorks:** support to wide range of HW, including powerful microprocessors
- **Xenomai/RTAI/...:** known as real-time linux, for more powerful HW. Actually they run Linux as a low-priority task

# To recap ....

- Embedded and Real-Time in perspective
- Basic definitions related with Real-Time
- Computational models (real-time model)
- Task characterization
  - Real-time tasks: periodic, sporadic and aperiodic
  - Event-triggered and time-triggered systems
- The task states
  - States and transition diagram
- The generic architecture of a RT kernel
  - The basic components of a RT kernel, its structure and functionalities
- Examples of RTOS